

Detector de Matrícules

Universitat Politècnica de Catalunya
Facultat d'Informàtica de Barcelona

Visió per Computador

2025-2026 Q2

VC Grup: 11

Adrià Jara Palos: adria.jara@estudiantat.upc.edu

Eric Medina León: eric.medina@estudiantat.upc.edu

Índex

1	Introducció	1
2	Descripció General del Funcionament	1
3	Implementació	3
3.1	Preprocessament i Generació de Candidats	3
3.1.1	Funcions Implementades	3
3.1.2	Decisions de Disseny	3
3.2	Classificador de Matrícules	4
3.2.1	Generació del Dataset	5
3.2.2	Funcions Implementades	5
3.2.3	Decisions de Disseny	6
3.2.4	Resultats i rendiment del classificador	7
3.2.4.1	Errors trobats	8
3.3	Neteja de Candidats Finals	8
3.3.1	Funcions Implementades	8
3.3.2	Funcionament del NMS	8
3.3.3	Mètriques de solapament i Criteris de Supressió	9
3.3.4	Decisions de Disseny i Resultats	9
3.4	Alineació de la Matrícula Candidata	9
3.4.1	Funcions Implementades	9
3.4.2	Decisions de Disseny	10
3.5	Segmentació de Caràcters	10
3.5.1	Funcions Implementades	10
3.5.2	Decisions de Disseny	11
3.6	OCR	13
3.6.1	Dataset Utilitzat	13
3.6.2	Funcions Implementades	13
3.6.3	Decisions de Disseny	14
3.6.4	Alternatives Descartades	15
3.6.5	Resultats i rendiment del classificador	15
3.7	Resum del Flux i generació d'output	16
3.7.1	Paràmetres de control	16
3.7.2	Pipeline	17
4	Resultats Finals	17
4.1	Extractes de la sortida	17
4.1.1	Anàlisi de Confusions Tipogràfiques	18
4.2	Exemples d'Execució	19
4.3	Alguns Errors Trobats	22
4.4	Conclusió	23
5	Annexos	24
5.1	Codi Font	24
5.1.1	main	24

5.1.2	candidates_detection	30
5.1.3	plate_classifier	32
5.1.4	character_segmentation	36
5.1.5	dataset_generation	39
5.1.6	ocr_classifier	43

1 Introducció

El reconeixement automàtic de matrícules (*Automatic License Plate Recognition*, ALPR) és una aplicació clàssica dins l'àmbit de la visió per computador que permet identificar vehicles a partir de les imatges capturades per càmeres. Aquest tipus de sistemes tenen nombroses aplicacions pràctiques, com ara el control d'accessos, la gestió d'aparcaments, la monitorització del trànsit o els sistemes de vigilància viària.

L'objectiu d'aquesta pràctica és desenvolupar un sistema complet de detecció i reconeixement de matrícules utilitzant tècniques de visió per computador apreses a classe. El sistema implementat consta de diverses etapes consecutives: la detecció de possibles regions d'interès que poden contenir una matrícula, la classificació d'aquestes regions per descartar falsos positius, l'alineació geomètrica de la matrícula detectada, la segmentació dels caràcters individuals i, finalment, el reconeixement de cada caràcter.

Totes les funcions són de creació pròpia i no han sigut agafades de fonts externes.

2 Descripció General del Funcionament

El sistema desenvolupat segueix una arquitectura seqüencial formada per diverses etapes de processament, cadascuna de les quals té com a objectiu reduir progressivament la informació de la imatge fins a obtenir el text corresponent a la matrícula. La Figura 1 mostra de manera esquemàtica el flux general de processament implementat.

En primer lloc, a partir de la imatge original del vehicle es realitza una fase de detecció de regions candidates. L'objectiu d'aquesta etapa és localitzar totes les zones que podrien correspondre a una matrícula. Per fer-ho, s'utilitzen tècniques de processament d'imatge basades en la detecció de contorns i propietats geomètriques dels objectes presents a la imatge. Aquesta fase prioritza obtenir una elevada taxa de detecció, encara que això impliqui generar un cert nombre de falsos positius.

Les regions candidates obtingudes són posteriorment analitzades per un classificador binari (SVM) encarregat de determinar si una determinada regió conté una matrícula o no. Aquest classificador ha estat entrenat utilitzant descriptors visuals extrets de les imatges i permet descartar la major part de les deteccions incorrectes. Posteriorment, s'aplica una etapa de *Non-Maximum Suppression* (NMS) per eliminar deteccions redundants que corresponen a una mateixa matrícula.

Una vegada identificada la regió més probable, es procedeix a la seva alineació geomètrica. Aquesta etapa corregeix possibles inclinacions o deformacions produïdes per la perspectiva de la càmera, amb l'objectiu d'obtenir una imatge de la matrícula el més frontal possible.

A continuació es realitza la segmentació dels caràcters. En aquesta fase la imatge de la matrícula es converteix a escala de grisos i es binaritza. Posteriorment s'apliquen diverses operacions morfològiques, com ara eliminació de soroll, tancament morfològic i filtratge per propietats geomètriques, amb la finalitat d'obtenir les regions corresponents als caràcters individuals. Els objectes detectats són filtrats segons criteris com la seva alçada relativa, relació d'aspecte i ocupació de la caixa contenidora. En cas de que la quantitat de caràcters detectats sigui inferior o igual a 3, es torna a realitzar l'alineació i segmentació amb la inversió de la imatge per intentar detectar matrícules amb fons negre i caràcters clars.

Una vegada segmentats els caràcters, es duu a terme el reconeixement de cadascun d'ells. Per representar cada símbol s'extreu un conjunt de característiques descriptives que inclouen descriptors HOG, perfils de projecció horitzontals i verticals, descriptors de densitat per zones, informació topològica i altres característiques estructurals. Aquest vector de característiques és introduït en un classificador basat en SVM, entrenat prèviament amb exemples de lletres i números.

Finalment, els caràcters reconeguts es concatenen seguint el seu ordre espacial dins de la matrícula per obtenir el text final detectat. Aquest resultat és comparat amb les anotacions disponibles al conjunt de dades per tal d'avaluar el rendiment global del sistema.

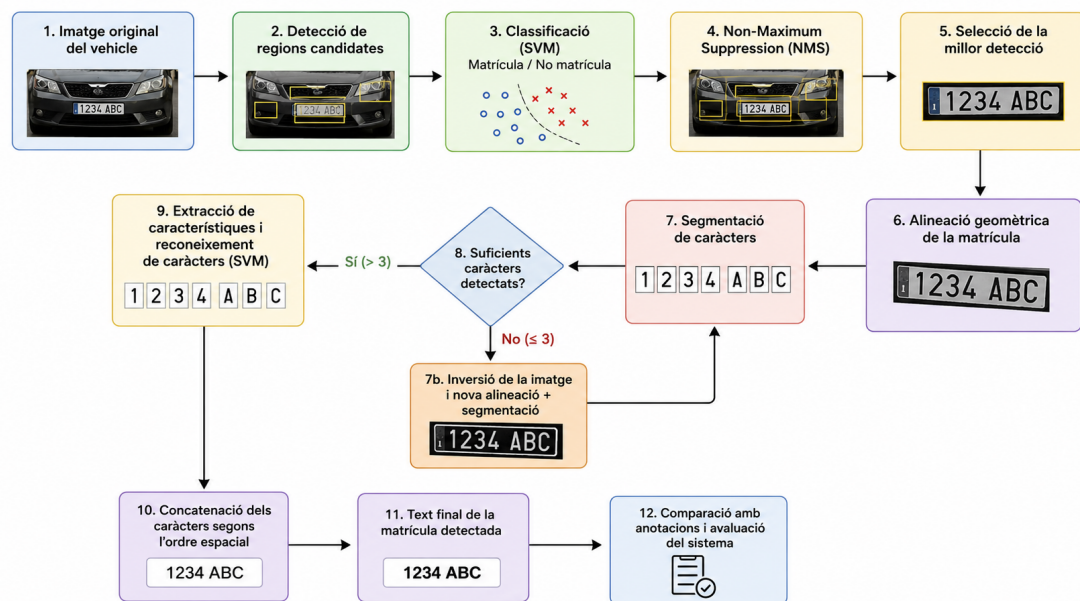


Figura 1: Esquema del pipeline de processament implementat per al reconeixement de matrícules.

3 Implementació

3.1 Preprocessament i Generació de Candidats

3.1.1 Funcions Implementades

Per a la detecció de les possibles regions d'interès (ROI) candidates a ser matrícules, s'ha implementat la funció `getCandidates`, que processa la imatge segons les següents etapes:

- **Preparació de la imatge:** Es converteix la imatge a escala de grisos (`rgb2gray`) i millorem el contrast amb `imadjust`
- **Detecció de vores:** S'aplica un filtre d'extracció de vores basat en Sobel, configurat per detectar exclusivament els gradients verticals (`edge(im, 'sobel', 'vertical')`).
- **Processament morfològic iteratiu:** S'itera sobre un conjunt d'elements estructurants rectangulars de diferents mides. Per a cadascun, es realitza un tancament (*close*) manual estructurat en tres subetapes: una dilatació, un ompliment de forats interns, i finalment la erosió.
- **Extracció de regions:** S'extreuen els components connexos amb `bwconncomp` i se n'obtenen les propietats geomètriques (`BoundingBox`, `Area`, `Extent`) a través de `regionprops`. A cada boundingbox resultant, se li aplica un petit marge de seguretat de píxels per no tallar la matrícula.
- **Filtratge de candidats:** Les regions extretes passen per un filtre final on es calculen paràmetres derivats per descartar falsos positius evidents abans de retornar els resultats.

3.1.2 Decisions de Disseny

El mètode que hem descrit prèviament, respon a una estratègia on busquem maximitzar el *recall*, assegurant-nos que la matrícula real sempre estigui entre els candidats. Deleguem la tasca d'eliminar els falsos positius al classificador.

Disseny del processament morfològic: S'ha escollit el filtre Sobel vertical perquè les matrícules generen una alta densitat de edges verticals a causa dels caràcters. A més, s'ha optat per una operació *close* manual per poder fer servir imfill i tancar agressivament els forats interns entre les lletres, fusionant la matrícula en un sol *blob* sòlid.

Per garantir la invariància a l'escala, hem optat per aplicar sis elements estructurants diferents.

- Mides fixes: `[1, 5]`, `[2, 12]`, `[5, 20]`, `[8, 30]`, `[12, 45]`
- Mida dinàmica: `[0.010 × sizeY, 0.030 × sizeX]` per adaptar-se a resolucions extremes de càmera.

Criteris de filtratge geomètric

S'han establert tres condicions de tall molt permissives per eliminar només les ROIs inassumibles:

- **isWide:** Limitem l'aspect ratio de la regió entre 1.7 i 12. Aquest marge tan ampli evita descartar matrícules que es vegin extremadament estirades a causa de la distorsió per perspectiva.
- **isRect:** Exigim un *Extent* (densitat de píxels del blob dins el bounding box) ≥ 0.15 per descartar soroll aïllat o línies extremadament fines.
- **isReasonable:** Es limita l'àrea relativa del blob. S'eliminen sorolls petits ($< 0.05\%$) i blobs que ocupin més d'un 25% de la imatge total. Assumim el risc teòric de descartar matrícules que ocupin tota la pantalla (poc comú en un entorn real) per evitar errors severs on es selecciona la carrosseria sencera del vehicle com a candidata (ex. *test_097*).

Alternatives Descartades: Durant la fase de disseny, es van avaluar, implementar, i descartar altres estratègies tradicionals per a la cerca de matrícules. La taula 1 resumeix els motius pels quals es van abandonar a favor de l'enfocament actual.

Mètode	Motivació	Motiu de descart
Detecció de Vèrtex	Ideal per trobar quatre punts que representen el contorn rectangular de la matrícula	Explosió combinatòria. S'obtenien situacions amb més de 150 vèrtexs candidats, representant milions de combinacions possibles (inassumible computacionalment). A baixa resolució rarament es detecten bé les quatre cantonades.
SIFT	Detecció directa basada en punts d'interès (POI) i pattern matching.	Les diferències entre els caràcters de cada matrícula confonen l'algorisme general. És incapaç de trobar un patró universal robuts.
Transformada de Hough	Detecció de línies rectes verticals i horitzontals.	En condicions de baixa il·luminació o alta distorsió, els contorns no es detectaven correctament o perdien prioritat davant d'altres línies rectes del vehicle.

Taula 1: Comparativa de mètodes de detecció de ROIs

3.2 Classificador de Matrícules

En aquesta etapa del projecte s'ha implementat un classificador binari encarregat de determinar si una regió d'imatge (ROI) conté o no una matrícula de vehicle. Cada

imatge es transforma en un vector de característiques i posteriorment es classifica mitjançant un model (SVM) prèviament entrenat.

3.2.1 Generació del Dataset

Per entrenar el classificador s'ha utilitzat el dataset públic *Car License Plate Detection* disponible a Kaggle, el qual conté 433 imatges de vehicles amb les corresponents anotacions en format XML, que inclouen la posició de la matrícula dins de cada imatge.

Per tenir les dades en un format adequat per a l'entrenament del model, s'ha realitzat un procés de generació de mostres positives i negatives a partir de les imatges anotades mitjançant un script. L'script realitza les següents operacions per a cada imatge del dataset:

- Lectura de la imatge i del fitxer XML associat.
- Extracció de la bounding box de la matrícula.
- Aplicació d'un padding del 5% per incloure context visual al voltant de la matrícula.
- Generació d'un exemple positiu mitjançant el retall directe de la matrícula.
- Generació de 5 exemples negatius mitjançant mostreig aleatori de regions de la imatge que no solapen amb la matrícula.

Aquests retalls són emmagatzemats en dues carpetes separades: una per a les mostres positives (matrícules) i una altra per a les mostres negatives (fons o objectes no rellevants).

Per garantir la qualitat dels exemples negatius, es calcula la mètrica Intersection over Union (IoU) entre la regió generada i la matrícula real. Només es conserven aquells crops amb IoU inferior a 0.1, evitant així mostres amb informació parcial de la matrícula.

3.2.2 Funcions Implementades

Les principals funcions implementades en aquest mòdul són:

- `compute_iou`: funció exclusiva de la part de generació del dataset. Donades dues bounding boxes, calcula la intersecció sobre la unió (IoU) d'aquestes per assegurar que els exemples negatius no solapen amb la matrícula. D'aquesta forma es garanteix que el classificador aprengui a diferenciar entre matrícules i fons.
- `extract_plate_features`: funció compartida entre la part d'entrenament i la part de classificació. Extreu el vector de característiques de cada imatge (explicades al següent apartat).

-
- **classify_roi**: funció encarregada de carregar el model SVM entrenat i extreure les característiques i classificar una regió d'interès (ROI) donada. Aquesta funció s'utilitza durant la fase de detecció per filtrar les regions candidates obtingudes a partir de les tècniques de processament d'imatge del mòdul anterior.

A part d'aquestes funcions principals, s'han implementat els scripts que gestionen tot el procés:

- **generate_dataset**: script encarregat de processar les imatges anotades i generar els crops positius i negatius (com explicat en l'apartat anterior).
- **train_plate_classifier**: script que extreu les característiques de cada imatge amb la funció **extract_plate_features**, divideix els conjunts de features y labels en conjunts de entrenament (80%) i test (20%), i entrena un model SVM amb kernel lineal. Finalment calcula i mostra les mètriques d'avaluació (accuracy, precision, recall i matriu de confusió) sobre el conjunt de test.

Totes aquestes funcions i scripts es poden trobar a l'annex.

3.2.3 Decisions de Disseny

Primer de tot, en les decisions de disseny del dataset d'entrenament, s'ha optat per buscar un d'internet ja que el dataset donat per l'entrega era massa petit (només 100 imatges) per entrenar un model de classificació robust. A més, es va decidir generar exemples negatius a partir de les mateixes imatges del dataset per garantir que el model aprengué a diferenciar entre matrícules i fons en el mateix context visual (portes, carrers, llums, etc). També es va decidir aplicar un padding del 5% a les bounding boxes de les matrícules per incloure una mica de context visual al voltant d'aquestes.

En el disseny del classificador s'ha optat per un enfocament basat en *Support Vector Machines* (SVM) amb kernel lineal, degut a la seva bona capacitat de generalització en problemes de classificació binària amb vectors de característiques d'alta dimensió.

Pel que fa a la representació de les imatges, s'ha escollit els següents descriptors:

- **HOG (Histogram of Oriented Gradients)**: com les matrícules tenen una estructura visual molt característica (línies horitzontals i verticals, contrast entre caràcters i fons), els descriptors HOG són molt adequats per capturar aquesta informació. S'han utilitzat 9 orientacions i cel·les de 8x8 píxels.
- **Aspect ratio**: relació entre amplada i alçada de la regió, que ajuda a diferenciar matrícules de regions quadrades o irregulars.
- **Area total**: mida de la regió, que aporta informació addicional sobre l'escala de l'objecte.

Inicialment només es van utilitzar els descriptors HOG. Tot i tenir una bona performance inicial, després de les proves es va observar que el model, en moments puntuals detectava regions amb aspect ratio molt diferent al d'una matrícula (per exemple, regions molt quadrades o molt allargades). Per aquest motiu es van afegir les característiques d'aspect ratio i area total. No van ser de gran ajuda degut a la dimensionalitat de HOG respecte a aquestes dues característiques, però es van decidir mantenir.

Totes les imatges es redimensionen a una mida fixa abans de l'extracció de característiques per garantir consistència en els descriptors. Es va decidir utilitzar una mida de 128x64 píxels.

3.2.4 Resultats i rendiment del classificador

Per avaluar el rendiment del classificador SVM s'ha utilitzat un conjunt de test independent del conjunt d'entrenament (del 20%) i s'ha establert una seed fixa (42) per reproduir els resultats. La mètrica principal utilitzada ha sigut la matriu de confusió, d'on s'han tret també les mètriques de accuracy, precision i recall.

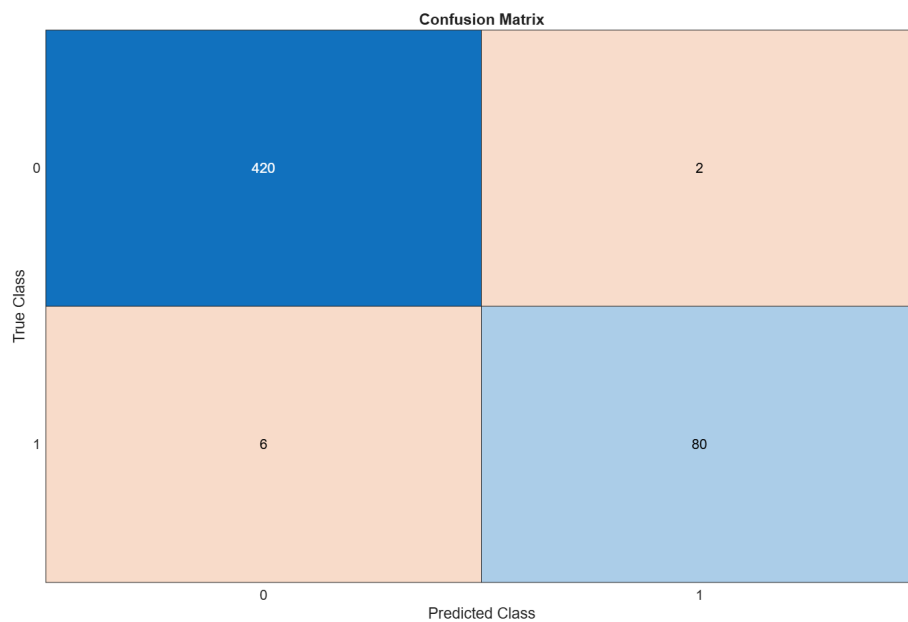


Figura 2: Matriu de confusió del classificador SVM

Després es van calcular les següents mètriques:

- **Accuracy:** 98.62%
- **Precision:** 97.56%
- **Recall:** 94.12%

Els resultats obtinguts mostren una capacitat de discriminació adequada entre les classes positives (matrícules) i negatives (no matrícules).

3.2.4.1 Errors trobats

Els principals errors es produeixen en regions amb característiques visuals similars a les matrícules, com ara parts del vehicle amb formes rectangulars (reixes, filtres, etc.) o text en el fons.

Tot i això, posteriorment es fa un tractament d'aquests casos a través que s'explica a continuació.

3.3 Neteja de Candidats Finals

Després de l'etapa de classificació de ROIs, el sistema genera habitualment múltiples candidats que corresponen a una mateixa matrícula.

Aquesta redundància provoca dos problemes principals:

- Increment innecessari del cost computacional en les fases posteriors (alineació i OCR).
- Possibles errors de reconeixement, ja que diferents crops de la mateixa matrícula poden produir resultats lleugerament diferents.

Per solucionar aquest problema s'ha implementat un procés de *Non-Maximum Suppression* (NMS), amb l'objectiu de conservar únicament el candidat més representatiu de cada regió de matrícula.

3.3.1 Funcions Implementades

Les funcions principals implementades en aquest mòdul són:

- **getOverlap**: funció encarregada de calcular el grau de solapament entre dues *bounding boxes*. A partir de dues regions definides com $A = [x, y, w, h]$ i $B = [x, y, w, h]$, es calcula IoU, el percentatge de la caixa A coberta per B i el percentatge de la caixa B coberta per A. A partir d'aquesta intersecció es deriven tres mètriques:
- **nms**: funció que implementa l'algorisme NMS, ordenant les caixes segons el seu score de classificació i eliminant les que presenten un solapament elevat amb altres caixes de major score.

3.3.2 Funcionament del NMS

L'algorisme NMS funciona ordenant inicialment totes les bounding boxes segons la seva probabilitat de ser matrícula (score retornat pel classificador SVM). A partir d'aquesta ordenació, es procedeix de la següent manera:

1. Es selecciona la bounding box amb major score com a referència.
2. Es comparen totes les altres caixes amb aquesta utilitzant mètriques de solapament.

-
3. Les caixes que presenten un solapament elevat són eliminades.
 4. Es repeteix el procés amb la següent caixa no eliminada.

3.3.3 Mètriques de solapament i Criteris de Supressió

Per determinar quan dues caixes representen el mateix objecte s'han utilitzat tres criteris:

- **Intersection over Union (IoU)**: mesura la relació entre l'àrea d'intersecció i la unió de dues caixes. Si $\text{IoU} \geq 0.45$, es considera que les dues regions representen el mateix objecte.
- **Containment ratio**: es calcula la proporció d'una caixa continguda dins de l'altra. Aquest criteri permet detectar casos en què una ROI està gairebé completament dins d'una altra.
- **Simetria de conteniment**: es considera tant la cobertura de la caixa A sobre B com de B sobre A, amb un llindar de 0.75.

Si es compleix qualsevol de les condicions següents, una de les dues caixes es descarta:

- $\text{IoU} \geq 0.45$
- $\text{Containment ratio} \geq 0.75$

En aquests casos, es manté sempre la caixa amb major score de classificació, eliminant les altres per evitar redundància.

3.3.4 Decisions de Disseny i Resultats

Inicialment, aquesta neteja de candidats es va implementar d'una forma més simple, sense seguir el criteri de supració de no màxims. En aquest primer enfocament, es van eliminar totes les caixes que presentaven un solapament superior a un cert llindar. Aquest mètode no va resultar ser suficient ja que en molts casos es seguien conservant múltiples caixes i no garantitzava que es mantingués la caixa amb major score. Per aquest motiu es va investigar i optar per implementar l'algorisme de NMS.

L'aplicació de NMS ha permès reduir significativament el nombre de candidats per imatge, eliminant duplicats de la mateixa matrícula i millorant l'estabilitat del sistema. A més, permet la detecció de múltiples matrícules en la mateixa imatge, ja que només es suprimeixen les caixes que representen el mateix objecte.

3.4 Alineació de la Matrícula Candidata

3.4.1 Funcions Implementades

Un cop obtingudes les regions candidates ya classificades, s'ha implementat una funció per alinear-les: `alignPlate`

-
- **Preparació de la imatge:** La imatge es converteix a escala de grisos, se n'ajusta el contrast, i es binaritza per llindar adaptatiu (`imbinarize(im, 'adaptive')`) per mitigar els canvis d'il·luminació.
 - **Càlcul de l'angle:** S'extreuen els components connexos de la imatge binaritzada i es selecciona el més gran. S'utilitza la propietat *Orientation* de `regionprops` per obtenir l'angle d'inclinació d'aquest component.
 - **Rotació i retall:** La imatge original i la binaritzada es roten utilitzant l'angle. Seguidament, es torna a calcular la *bounding box* sobre la imatge ja recta per eliminar l'espai negre residual generat per la rotació, i fer un retall definitiu. Aquest pas és per acotar més la matrícula doncs en alguns casos la ROI seguia sent massa gran.
 - **Projecció geomètrica:** Es fa servir una matriu de rotació 2D tenint en compte els centres geomètrics de la imatge rotada i original per projectar els quatre vèrtexs de la nova *bounding box* cap a les coordenades absolutes de la imatge original. Aquest últim pas només afecta a la visualització de la regió.

3.4.2 Decisions de Disseny

L'objectiu d'aquesta etapa és corregir les distorsions de perspectiva i inclinació de la càmera, ja que els algorismes de reconeixement de caràcters (OCR) i la extracció de característiques són sensibles a la rotació.

Per aquesta raó, hem escollit la solució amb components connexos amb `regionprops` doncs és un mètode robust per trobar la inclinació i dades de la matrícula. En la majoria de casos, la matrícula és el *blob* més gran de la ROI.

Alternatives Descartades: Durant la fase de disseny, es van avaluar, implementar i descartar un altre estratègia tradicional per alinear les matrícules.

- **Transformada de Hough:** Mitjançant Canny i la transformada de Hough, es va crear un mètode similar al actual, però els resultats no eren els esperats, i algunes matrícules no tenien la rotació correcta amb un error inassumible. Per aquest motiu, vam descartar aquesta implementació a favor de la actual.

3.5 Segmentació de Caràcters

3.5.1 Funcions Implementades

Un cop la matrícula ha estat retallada i alineada, la funció `segm` s'encarrega d'aïllar i extreure cadascun dels caràcters presents a la imatge per al seu posterior reconeixement. El procés es divideix en les següents fases:

- **Preparació de la imatge:** La imatge es converteix a escala de grisos, s'hi aplica un filtre gaussià per difuminar el soroll, es binaritza invertint el resultat.
- **Neteja del fons:** S'eliminen tots els objectes que toquen els marges de la imatge i s'esborren els blobs d'àrea minúscula.

-
- **Filtratge de components connexos:** Obtenim els components connexos de la imatge i apliquem un filtre sobre les regions. Les condicions estan documentades a **Geometria Estricta**.
 - **Mecanisme d'homogeneïtat:** Si s'han detectat tres o més candidats validats, es calcula la mediana de les seves alçades. Tots aquells candidats on l'alçada sigui inferior al 80% de la mediana, són descartats.
 - **Ordenació final:** Ordenem els caràcters que han passat el filtratge per posició d'esquerra a dreta, per tindre l'ordre de la matrícula i facilitar la següent fase.

3.5.2 Decisions de Disseny

L'etapa de segmentació requereix un equilibri molt precís: ha de ser prou estricta per no passar blobs de brutícia o soroll, logotips, escuts, etc al classificador, però prou permissiu per no descartar lletres en matrícules lleugerament tapades o deteriorades.

Estratègia de binarització: Usem un filtre gaussià amb $\sigma = 0.5$ prèviament per eliminar soroll i poder realitzar la binarització adaptativa sense problemes. Un cop binaritzat, netegem vores i eliminem petits sorolls. **Geometria Estricta:** Els valors límit per els blobs obtinguts amb **regionprops** són:

- **Aspect Ratio:** 0.05 fins a 1.5 per incorporar caràcters anòmals.
- **Alçada:** D'un 35% a un 95%, en les matrícules ja retallades, és difícil que un caràcter sigui més baix que el límit inferior.
- **Extent:** 0.12 fins a 1.0. Els caràcters no són blocs sòlids, però tenen una densitat mínima, afegim 0.12 com a límit inferior per caràcters com l'**L**.

Filtratge relatiu per Mediana: Tot i el filtratge geomètric, hi han alguns blobs corresponents a figures com la franja de la UE, escuts, etc que passen el filtre. Per pal·liar-ho, vàrem incorporar un filtre de context: es comparen les alçades de tots els candidats respecte a la seva pròpia mediana. Com que en una matrícula els caràcters solen ser pràcticament de la mateixa mida, això elimina la major part dels falsos positius.

Alternatives Descartades: Durant el desenvolupament de la segmentació, es van explorar tècniques alternatives que finalment es van descartar pels motius exposats a la taula 2.

Mètode	Motivació	Motiu de descart
Template Matching	Buscar proporcions exactes de caràcters dins de la regió binaritzada.	Incapacitat d'adaptar-se a les lletres deformades pel soroll i baixes resolucions.
Mètode actual amb aspect ratio ≥ 0.95	Restringir l'amplada dels caràcter seguint l'estàndard europeu	Al no ser totes les matrícules del mateix país, en alguns casos hi havien caràcters més llargs que alts.
Projecció Vertical i Horitzontal	Segons la nostra recerca, és un mètode usat en OCR per separar caràcters.	Les matrícules amb lleugera rotació, o perspectiva feien que els caràcters no es detectessin correctament.

Taula 2: Comparativa de mètodes de segmentació avaluats prèviament.

Exemple d'errors de segmentació: En la figura 3 podem veure un clàssic error de segmentació en el nostre programa, l'1 final no s'ha segmentat correctament ja que, al binaritzar, queda com un únic blob juntament amb la franja de color negre per l'alineament de la placa. Per corregir-ho, vàrem sospesar dues opcions

- Ampliar els marges de la placa passada a la segmentació
- Al girar, fer servir una màscara i assignar el valor 1 (blanc) als píxels afegits per l'alineament.

Totes dues solucions van fer que baixés el percentatge de matrícules correctes i vam optar per no solucionar-ho i assumir l'error.

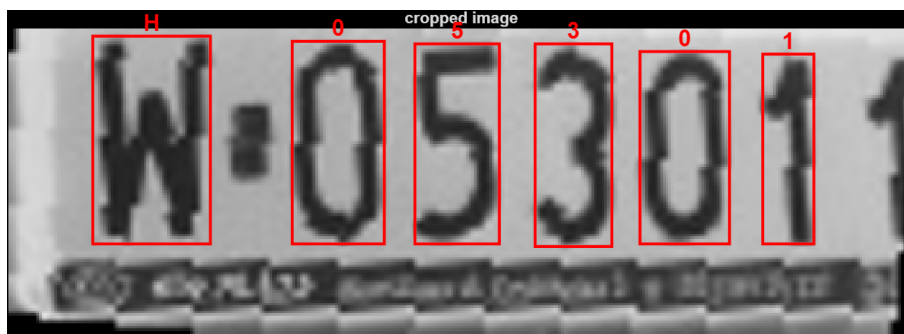


Figura 3: Caràcter mal retallat

Per les següents figures, podem veure el dilema que hem tingut: Aplicar una solució, perjudica un altre imatge. En la figura 4 podem veure com el caràcter **5** es fusiona amb la part baixa de la matrícula. Si apliquessim un *close* per eliminar l'espai que uneix el **5** amb la banda, ens trobaríem que, amb imatges com la de la figura 5 hi han caràcters que es trenquen, com el cas de la **W**, el caràcter es trenca, i queden dos blobs independents, dels quals un s'ha descartat.

Amb aquest problema en ma, hem decidit no aplicar l'operador morfològic, ja que ens ha donat millor resultats sobre el ground truth.

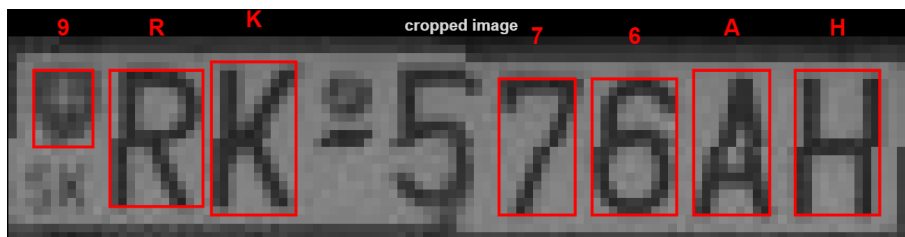


Figura 4: Caràcter fusionat amb vores

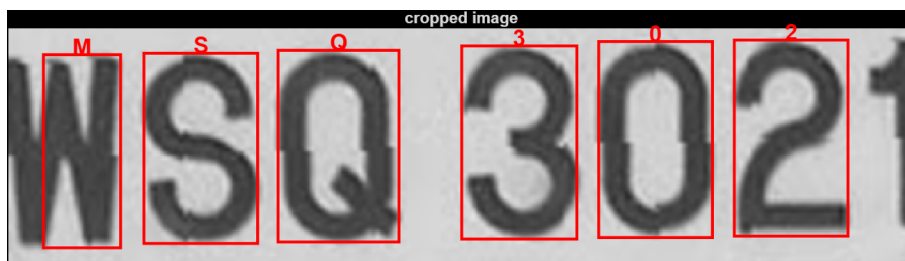


Figura 5: Caràcter parcialment segmentat

3.6 OCR

En aquesta etapa del sistema s'ha implementat un classificador OCR (SVM) encarregat d'identificar cada caràcter individual segmentat de la matrícula (lletres i números). A partir dels blobs obtinguts en la fase de segmentació, el sistema assigna una etiqueta a cada regió mitjançant un model entrenat prèviament.

3.6.1 Dataset Utilitzat

Per a l'entrenament del classificador OCR s'ha utilitzat un dataset públic d'OCR disponible a Kaggle (**standard OCR dataset**), el qual conté imatges de caràcters individuals (lletres i números) organitzades en carpetes separades per classe. Cada classe conté aproximadament entre 500 i 600 imatges, proporcionant una distribució equilibrada entre caràcters.

El dataset s'ha carregat utilitzant un `imageDatastore` de MATLAB, configurat per llegir automàticament les subcarpetes i assignar les etiquetes segons el nom de cada carpeta.

3.6.2 Funcions Implementades

Les principals funcions implementades en aquest mòdul són:

- `extract_character_features`: funció compartida entre l'entrenament i la part de classificació. S'encarrega d'extreure el vector de característiques de cada caràcter (explicades al següent apartat).

-
- **classify_character**: funció que carrega el model SVM entrenat, extreu les característiques i retorna la predicció del caràcter corresponent a una imatge d'entrada.
 - **recognize_plate**: funció que recorre tots els blobs detectats en una matrícula segmentada, aplica el classificador (`classify_character`) a cada regió i construeix la cadena final de la matrícula concatenant les prediccions.

A part d'aquestes funcions principals, s'ha implementat el script que gestiona l'entrenament del model:

- **train_ocr_classifier**: script que carrega el dataset d'OCR, divideix els conjunts de features y labels en conjunts d'entrenament (80%) i test (20%), extreu les característiques de cada imatge amb la funció `extract_character_features` i entrena un model SVM amb kernel lineal. Finalment calcula i mostra les mètriques d'avaluació (accuracy i matriu de confusió) sobre el conjunt de test.

3.6.3 Decisions de Disseny

Primer de tot, respecte la selecció del dataset d'entrenament, s'han utilitzat diverses alternatives al llarg del procés. Inicialment es va utilitzar un altre dataset (també d'internet) el qual contenia imatges de caràcters directament obtingut de matrícules reals. Tot i això, es van observar diversos problemes després de fer les primeres proves. Les causes indentificades van ser que totes les imatges (de cada caràcter) eren de la mateixa tipografia, per tant, el model no era capaç de generalitzar a altres fonts d'imatges de forma correcta. A més, moltes estaven rotades o deformades, i en el nostre cas això perjudicava més que ajudava ja que en el nostre pipeline es realitza una etapa d'alineació prèvia a la segmentació, per tant, els caràcters sempre apareixien rectes (sense rotacions exagerades). Després de descartar aquest dataset, es va optar per utilitzar el dataset d'OCR clàssic mencionat anteriorment.

Per a la representació dels caràcters s'han combinat múltiples descriptors amb l'objectiu de capturar tant la forma global com les estructures internes dels símbols:

- **HOG (Histogram of Oriented Gradients)**: utilitzat amb cel·les de 4x4 píxels per capturar millor els detalls fins dels caràcters, com traços i cantonades. Inicialment es van provar cel·les de 8x8, però va reduir a 4x4 degut a la mida reduïda de les imatges.
- **Densitat global**: proporció de píxels actius dins la imatge binaritzada, que ajuda a diferenciar caràcters plens de caràcters oberts.
- **Nombre de components connectats**: mesura del nombre de components connectats de la imatge binaritzada negada, que pot ajudar a distingir caràcters amb forats (per exemple B i 8).
- **Perfils de projecció**: distribució horitzontal i vertical dels píxels, normalitzada a longitud fixa, que permet capturar l'estructura general del caràcter.

-
- **Zoning 4x4 i 6x3:** divisió de la imatge en regions per calcular densitat local, aportant informació espacial sobre la distribució dels traços.
 - **Crossing numbers:** nombre de transicions entre fons i primer pla per fila, útil per distingir caràcters amb estructures similars.

Inicialment només es van utilitzar els descriptors HOG però es va observar que el model tenia dificultats per diferenciar entre caràcters amb formes similars (per exemple, 0 i O, o 1 i I). Per això es van afegir tots els descriptors addicionals de cop, per intentar aportar més informació i compensar la dimensionalitat de les característiques HOG.

Totes les imatges es redimensionen a una mida fixa de 32x32 píxels abans de l'extracció de característiques per garantir consistència entre mostres.

3.6.4 Alternatives Descartades

Durant el procés d'implementació del classificador OCR es van provar diverses alternatives que finalment es van descartar per diferents motius:

- **Augmentar la mida de les imatges:** es va considerar redimensionar les imatges a una mida més gran (per exemple, 64x64) per capturar més detalls, però es va descartar aquesta opció ja que va incrementar significativament el temps d'entrenament i no va aportar millores substancials en el rendiment del model.
- **Dataset amb caràcters de matrícules reals:** com s'ha mencionat anteriorment, es va provar un dataset amb imatges de caràcters obtinguts directament de matrícules reals, però es va descartar degut a la falta de varietat en les tipografies i a la presència de rotacions i deformacions que no eren representatives del nostre pipeline.
- **Augmentar la mida del dataset:** es va considerar fusionar diversos datasets per obtenir una major varietat de caràcters i tipografies, però no va resultar efectiu.
- **Finestra de 8x8 en HOG:** es va provar utilitzar una finestra de 8x8 píxels per a l'extracció de característiques HOG, però es va observar que aquesta configuració no capturava suficients detalls dels caràcters.

3.6.5 Resultats i rendiment del classificador

El rendiment del model s'ha avaluat utilitzant un conjunt de test independent (20% del dataset total), amb una divisió aleatòria de les dades. La mètrica principal utilitzada ha sigut la matriu de confusió, d'on s'han tret també la mètrica de accuracy.

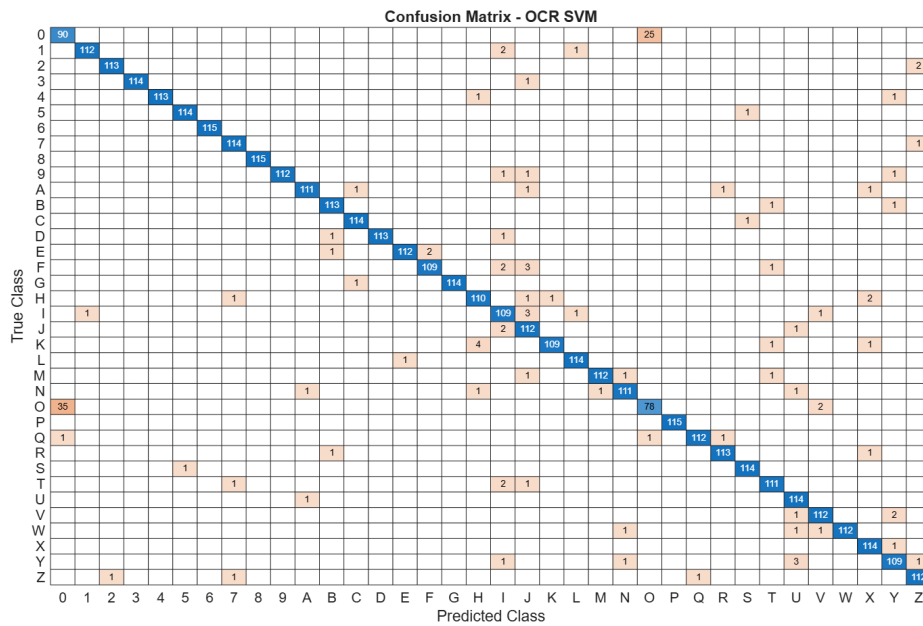


Figura 6: Matriu de confusió del classificador OCR

- Accuracy: 96.72%

Com es pot observar a la matriu de confusió, el model té una bona capacitat de discriminació entre les diferents classes de caràcters, tot i que hi han casos concrets on li costa diferenciar depèn de quin caràcter. On més es pot identificar aquest problema és entre el 0 i la O, amb una quantitat considerable de confusions. Els demès errors es distribueixen de forma més dispersa i puntual.

3.7 Resum del Flux i generació d'output

En aquest apartat resumim el programa principal, els seus paràmetres de control i les diferents etapes del nostre pipeline.

3.7.1 Paràmetres de control

A l'inici del programa es poden trobar les següents variables de control:

- **loadDataset:** Variable de control per carregar directament les imatges de la carpeta "Datasets/Dataset". **useTxtFiles** te preferència sobre aquesta variable, i la informació final és incompleta al no tindre dades dels arxius del ground truth.
- **useTxtFiles:** Carrega els arxius ".txt" de la carpeta "Datasets/Dataset". Permet extreure dades d'avaluació segons les posicions. Es recomana deixar a true.
- **fileFactor:** Percentatge [0.0, 1.0] d'arxius a carregar. Per els casos on només es vol executar una part petita del dataset.

-
- **showVisuals:** Variable per ensenyar les imatges resultants amb les ROI i la segmentació, o només sortida de consola.

3.7.2 Pipeline

Un cop carregades les imatges, el pipeline per a cada imatge seqüencial és el següent:

- **Fase 1: Detecció i filtratge de ROIs** S'extreuen els candidats a ser ROI amb `getCandidates`. A continuació, cada candidat passa a un classificador `classify_roi` que descarta ls crops que no encaixen amb una matrícula. Finalment, s'aplica un algorisme de *Supressió de No Màxims* (NMS) sobre les prediccions vàlides per eliminar bounding boxes solapades, retenint la que és més probable que sigui una matrícula.
- **Fase 2: Alineació i Reconeixement:** Sobre els candidats que han superat la fase anterior, apliquem una correcció de perspectiva amb la funció `alignPlate` seguit d'un escalat per millorar la resolució. Seguidament, passem el crop a la funció `segm` i aquesta ens retorna un conjunt ordenat de blobs, que passem al classificador de caràcters.
- **Mecanisme de recuperació (Fallback):** Si la quantitat de caràcters que hem detectat dins la ROI alineada és inferior a 3, assumim que pot existir un problema d'il·luminació o contrast (el cas més habitual és caràcters blanc sobre fons fosc) En aquests casos, invertim el color de la imatge, i repetim el procés sencer de la fase 2 per aconseguir una bona segmentació.
- **Recol·lecció de dades:** Un cop s'obté una lectura de matrícula, avaluem si és un match exacte, parcial o si hi ha error per les estadístiques. S'escriu per la consola al moment la matrícula esperada i la que hem obtingut amb el nostre pipeline.

Un cop acabades les fases, imprimim les dades recol·lectades durant la execució per tindre mètriques d'avaluació del nostre programa.

4 Resultats Finals

En aquest apartat presentem i analitzem els resultats globals obtinguts en executar el *pipeline* de detecció i reconeixement de matrícules sobre el dataset de prova, el qual compta amb un total de 108 imatges. El sistema ha assolit ua *tasa d'encert* del 63% (68 imatges sobre el total) i una *tasa d'encert parcial* (3 caràcters o més) del 98% (38 imatges sobre el total). Únicament s'han registrat 2 errors totals.

4.1 Extractes de la sortida

Aquí podem veure un extracte de la sortida final. Veiem els resultats que ja hem comentat. Les imatges que donen error han fallat en algun punt de la *Fase 1: Detecció i filtratge de ROIs*.

```

=====
RESULTADOS FINALES
Imagenes evaluadas: 108
=====
Iguales: 68
Parciales: 38
Errores: 2
=====
Tasa de acierto: 63%
Tasa de acierto parcial: 98%

```

4.1.1 Anàlisi de Confusions Tipogràfiques

Per tal d'identificar les debilitats del model de reconeixement de caràcters (OCR), hem extret les confusions més freqüents. Classifiquem els errors en tres blocs principals:

- **Ambigüïtat alfanumèrica** $0 \rightarrow 0$: L'error amb diferència més repetit, la única distinció és la distància de les vores al centre, però depèn fortament d'on procedeix la matrícula i de la resolució, doncs moltes vegades són detalls subtils que es perden.
- **Densitat de traços** $M \rightarrow N \wedge W \rightarrow H$: Els caràcters llargs com M i W presenten una alta densitat de línies de contrast en un espai a vegades molt reduït, fent que a vegades es puguin confondre amb una N o una H respectivament.
- **Deformació per soroll** $0 \rightarrow Q \wedge 8 \rightarrow B$: Aquests errors solen vindre per soroll o pèrdua de resolució al alinear la matrícula. Una petita traça fosca aherida a la part inferior dreta d'un 0 pot semblar una Q per al classificador. De la mateixa manera, la pèrdua de resolució pot fer que la part esquerra d'un 8 s'aplani lleugerament i el classificador ho entengui com una B.

Afegim una taula de referència amb els errors més repetits, obviant els errors que apareixen només un únic cop.

Caràcter Real	Caràcter Detectat	Nombre de vegades
0	0	7
M	N	5
0	Q	5
W	H	4
8	B	3
0	Q	3
W	H	3
A	N	2
W	N	2
V	Y	2

Taula 3: Confusions tipogràfiques més significatives

4.2 Exemples d'Execució

A continuació podem observar tres escenaris respresentatius del funcionament del sistema.

*Nota: les rois de color verd representen el ground truth i les de color vermell les trobades pel nostre sistema.



Figura 7: Cas correcte1

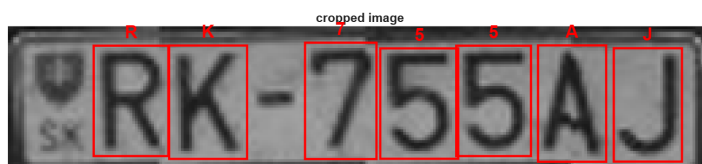


Figura 8: Caràcters detectats



Figura 9: Cas correcte2

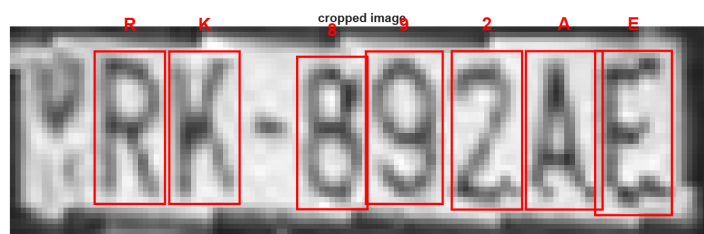


Figura 10: Caràcters detectats



Figura 11: Cas correcte3 (doble detecció)

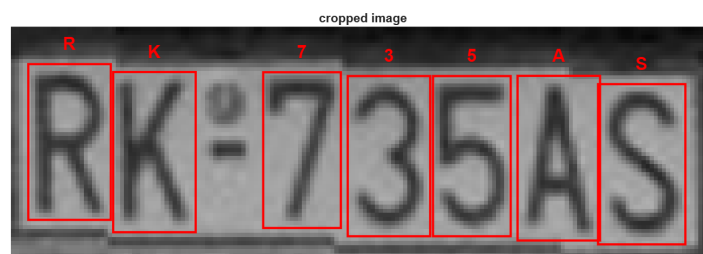


Figura 12: Caràcters detectats

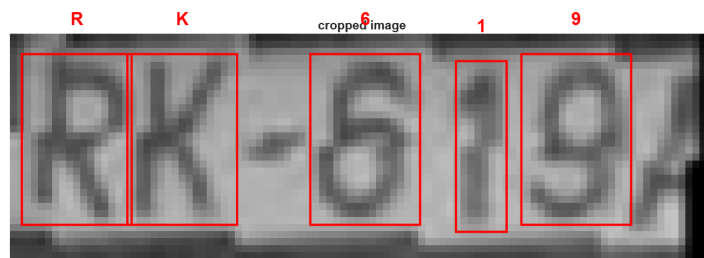


Figura 13: Caràcters detectats

4.3 Alguns Errors Trobats

A continuació es mostren alguns exemples d'imatges que han donat error.



Figura 14: Detecció extra incorrecta



Figura 15: Error de detecció

4.4 Conclusió

Com a conclusió d'aquesta fase d'avaluació, els resultats obtinguts demostren una alta robustesa en el disseny del pipeline, especialment pel que fa a les etapes de localització i alineació. L'assoliment d'una *taxa d'encert parcial del 98%* (106 de 108 imatges) confirma l'eficàcia dels filtres geomètrics. La inclusió del mecanisme de recuperació per inversió de color ens ha permès salvar situacions complexes de contrast i il·luminació que d'altra manera haurien resultat errors.

D'altra banda, creiem que la nostra *taxa d'encert del 63%* és millorable, començant per la segmentació, on per decisions de disseny en fases anterior, hem tingut que prendre decisions sobre com segmentar i que deixar com a possible error. Per judici visual, hem calculat que tenim un 87% de les imatges totals correctament segmentades, el qual ha condicionat diverses matrícules que podrien haver sigut correctes. De cara al classificador de caràcters, també ens ha perjudicat greument la baixa resolució de les imatges, les quals no hem pogut classificar correctament en casos com els mencionats a 3.

Amb l'objectiu de millorar el sistema en un futur, creiem que s'haurien d'implementar les següents estratègies:

- **Mida matrícula:** Donar més marge a les matrícules passades a la segmentació per no tindre problemes al netejar vores ni extreure soroll i millorar les operacions morfològiques.
- **Post-Processament basat en sintaxis:** Si coneixem els països d'origen de les matrícules, la classificació d'aquestes i aplicar regles segons el text pot ajudar. En el nostre cas, moltes matrícules seguien un patró i podríem solucionar aproximadament el 60% d'errors de confusió "0→O".
- **Augment de dades del classificador:** Estendre el dataset d'entrenament del classificador amb caràcters degradats, amb soroll i deformats.

5 Anexos

5.1 Codi Font

5.1.1 main

```
% PIPELINE COMPLETO DE DETECCIÓN Y RECONOCIMIENTO DE MATRÍCULAS

addpath(genpath(pwd));

function image = cutImage(im, ROI)
    image = imcrop(im, ROI);
end

% Flags para controlar la visualización y ejecución del sistema

showSteps = false;      % Muestra pasos intermedios del detector de ROIs
showSegments = true;     % Muestra segmentación de caracteres dentro de la ROI

loadDataset = false;
useTxtFiles = true;
fileFactor = 1;
showVisuals = true;

% Ruta del dataset
datasetPath = fullfile(pwd, 'Datasets', 'Dataset');

% Lista de imagenes a procesar
images = ["test_092.jpg", "test_071.jpg", "test_073.jpg", "test_061.jpg", "test_057.jpg"];

% Carga del dataset
if useTxtFiles
    disp('Using txt');
    files = dir(fullfile(datasetPath, '*.txt'));
    images = [];
    roisDataset = [];
    platesDataset = [];
    for f = 1:(numel(files) * fileFactor)
        file = fullfile(datasetPath, files(f).name);
        fid = fopen(file, 'r');
        data = textscan(fid, '%s %d %d %d %d %s');
        fclose(fid);

        if ~isempty(data{1})
            imgPath = fullfile(datasetPath, string(data(1)));
        end
    end
end
```

```

        x = data{2};
        y = data{3};
        w = data{4};
        h = data{5};
        pl = string(data{6});
        bb = [x, y, w, h];
        images = [images(:); imgPath(:)];
        roisDataset = [roisDataset; bb];
        platesDataset = [platesDataset; pl];
    end
end
elseif loadDataset
    images = [];
    data = [
        dir(fullfile(datasetPath, '*.jpg'));
        dir(fullfile(datasetPath, '*.png'));
        dir(fullfile(datasetPath, '*.jpeg'))
    ];

    images = fullfile({data.folder}, {data.name});
    images = string(images);

else
    images = fullfile(datasetPath, images);
    images = string(images);
end

% Bucle principal de procesamiento
exactas = 0;
parciales = 0;
errores = 0;

confusionLog = {};

for k = 1:numel(images)

    % Lectura de la imagen actual
    im = imread(images(k));
    if useTxtFiles
        expected = char(platesDataset(k));
        bestMatch = 0;
        bestPlate = "";
    end

    % FASE 1: DETECCIÓN DE POSIBLES MATRÍCULAS (ROIs)

```

```

candidates = getCandidates(im, false);

% Clasificar todas las ROIs y recoger scores ANTES del NMS
scores = zeros(size(candidates, 1), 1);
validMask = false(size(candidates, 1), 1);

for i = 1:size(candidates, 1)
    crop = cutImage(im, candidates(i,:));
    [isPlate, scores(i)] = classify_roi(crop);
    validMask(i) = isPlate;
end

% Quedarse solo con las que el clasificador acepta
candidates = candidates(validMask, :);
scores = scores(validMask);

% NMS sobre las candidatas válidas
candidates = nms(candidates, scores, 0.45, 0.75);

[~, name] = fileparts(images(k));
if showVisuals
    og = figure('Name', name);
    imshow(im), title(name);
    hold on;
end

% FASE 2: ALINEACIÓN Y SEGMENTACIÓN
for i = 1:size(candidates, 1)
    plate = cutImage(im, candidates(i,:));
    match = 0;

    [aligned, BB] = alignPlate(plate, candidates(i,:));
    aligned = imresize(aligned, 2);
    blobs = segm(aligned);

    [plateText, charLabels] = recognize_plate(aligned, blobs);
    fprintf("Matrícula detectada: %s\n", plateText);

    if size(blobs,1) <= 3
        [aligned, BB] = alignPlate(255-plate, candidates(i,:));
        aligned = imresize(aligned, 2);
        blobs = segm(aligned);
        [plateText, charLabels] = recognize_plate(aligned, blobs);
        fprintf("Matrícula detectada: %s\n", plateText);
    end
    fprintf("Matrícula detectada: %s\n", plateText);

```

```

detectedCh = char(plateText);

if useTxtFiles
    if strcmp(detectedCh,expected)
        match = 2;
    else
        m = 0;
        tmp = expected;
        for c = 1:length(detectedCh)
            idx = strfind(tmp, detectedCh(c));
            if ~isempty(idx)
                m = m + 1;
                tmp(idx(1)) = [];
            end
        end
        if m >= 3
            match = 1;
        end
    end
end

if match > bestMatch
    bestMatch = match;
    bestPlate = plateText;
end

end

% Visualización

if showVisuals
    figure(og);
    if useTxtFiles
        bbDataset = roisDataset(k, :);
        rectangle('Position',bbDataset, 'EdgeColor', 'g', 'LineWidth', 2.5)
    end

    if ~isempty(BB)
        x_pl = [BB(:,1); BB(1,1)];
        y_pl = [BB(:,2); BB(1,2)];
        line(x_pl, y_pl, 'Color', 'r', 'LineWidth', 2.5);

        if size(blobs,1) > 3 && showSegments
            figure, imshow(aligned), title('cropped image');
            hold on;
            for j = 1:size(blobs,1)
                bb = blobs(j,:);
                rectangle('Position', bb, ...

```

```

        'EdgeColor', 'r', ...
        'LineWidth', 2);
    text(bb(1)+bb(3)/2, ...
        bb(2)-5, ...
        char(charLabels(j)), ...
        'FontSize', 16, ...
        'FontWeight', 'bold', ...
        'Color', 'r', ...
        'HorizontalAlignment', 'center');
    end
    if showVisuals
        hold off;
    end
end
end
end
end

if showVisuals
    figure(og);
    hold off;
end

if useTxtFiles
    if bestMatch == 2
        exactas = exactas + 1;
        ver = "Correcto";
    elseif bestMatch == 1
        parciales = parciales + 1;
        ver = "Parcialmente correcto";

        % Analisis de Confusiones
        det = char(bestPlate);
        exp = expected;
        tmpExp = exp;
        tmpDet = det;
        % Elimina los caracteres que coinciden para quedarse solo con errores
        for c = length(tmpDet):-1:1
            idx = strfind(tmpExp, tmpDet(c));
            if ~isempty(idx)
                tmpExp(idx(1)) = [];
                tmpDet(c) = [];
            end
        end
        % Empareja lo que faltaba con lo que se detectó de más
        nPairs = min(length(tmpExp), length(tmpDet));

```

```

        for c = 1:nPairs
            confusionLog{end+1} = {tmpExp(c), tmpDet(c)};
        end
    else
        errores = errores + 1;
        ver = "Error";
    end
    fprintf("Archivo: %s\n", name);
    fprintf("Matrícula detectada: %s\n", bestPlate);
    fprintf("Matrícula esperada: %s\n", expected);
    fprintf("Veredicto: %s\n", ver);
    fprintf("=====\n");
end
if ~showVisuals
    close all hidden;
end
end

fprintf("\n=====\n");
fprintf("      RESULTADOS FINALES      \n");
fprintf("Imágenes evaluadas: %d\n", numel(images));
fprintf("=====\n");
fprintf("Iguales: %d\n", exactas);
fprintf("Parciales: %d\n", parciales);
fprintf("Errores: %d\n", errores);
fprintf("=====\n");
precision = (exactas / numel(images));
precParcial = ((exactas + parciales) / numel(images));
fprintf("Tasa de acierto: %.0f%%\n", precision * 100);
fprintf("Tasa de acierto parcial: %.0f%%\n", precParcial * 100);

% RESUMEN DE CONFUSIONES
if ~isempty(confusionLog)
    % Agrupa y cuenta cada par (real -> detectado)
    pares = cellfun(@(x) sprintf('%s->%s', x{1}, x{2}), confusionLog, 'UniformOutput', false);
    [paresSorted, ~, ic] = unique(pares);
    counts = accumarray(ic, 1);
    [countsSorted, sortIdx] = sort(counts, 'descend');

    fprintf("\n=== CONFUSIONES MÁS FRECUENTES ===\n");
    fprintf("  Real -> Detectado  Veces\n");
    fprintf("  -----\n");
    for i = 1:length(paresSorted)
        fprintf("  %-18s  %d\n", paresSorted{sortIdx(i)}, countsSorted(i));
    end
    fprintf("=====\n");
end

```

end

5.1.2 candidates_detection

```
function candidates = getCandidates(im, showSteps)
    im = rgb2gray(im);
    im = imadjust(im);
    [szy, szx] = size(im);
    if showSteps
        figure, imshow(im), title('input image');
    end
    candidates = [];

    area = numel(im);
    edges = edge(im, 'sobel', 'vertical');
    if showSteps
        figure, imshow(edges), title('edges');
    end
    sizes = [
        1, 5;
        2, 12;
        5, 20;
        8, 30;
        12, 45;
        ceil(0.010 * szy), ceil(0.030 * szx);
    ];
    for si = 1:size(sizes, 1)
        he = sizes(si, 1);
        wi = sizes(si, 2);
        EE = strel('rectangle', [he, wi]);
        EE2 = strel('rectangle', [he, wi]);

        cl = imdilate(edges, EE);
        cl = imfill(cl, 'holes');
        cl = imerode(cl, EE2);

        if showSteps
            figure, imshow(cl), title('close');
        end

        CC = bwconncomp(cl);
        S = regionprops(CC, 'BoundingBox', 'Area', 'Extent');

        for k = 1:numel(S)
            bb = S(k).BoundingBox;
```

```

        x = max(bb(1) - 10, 0);
        y = max(bb(2) - 10, 0);
        w = min(bb(3) + 20, szx);
        h = min(bb(4) + 20, szy);
        if w < 10 || h < 10
            continue;
        end

        ar = w / h;
        if ar < 1
            ar = 1 / ar;
        end

        areaRatio = S(k).Area / area;

        isWide = (ar > 1.7) && (ar <= 12);
        isRect = S(k).Extent >= 0.15;
        isReasonable = (areaRatio > 0.0005) & (areaRatio <= 0.25);
        if isWide && isRect && isReasonable
            candidates = [candidates; [x y w h]];
        end
    end
end
end

```

```

function kept = nms(candidates, scores, iouThresh, containThresh)

```

```

    if nargin < 4, containThresh = 0.75; end
    if nargin < 3, iouThresh = 0.45; end

    if isempty(candidates)
        kept = zeros(0, 4);
        return;
    end

```

```

    boxes = double(candidates);
    scores = double(scores);

```

```

    % Ordenar por score descendente
    [~, order] = sort(scores, 'descend');
    boxes = boxes(order, :);

```

```

    keep = true(size(boxes, 1), 1);

```

```

    for i = 1:size(boxes,1)-1

```

```

        if ~keep(i)
            continue;
        end
        for j = i+1:size(boxes,1)
            if ~keep(j)
                continue;
            end

            [iou, r1, r2] = getOverlap(boxes(i,:), boxes(j,:));

            if iou >= iouThresh || r1 >= containThresh || r2 >= containThresh
                keep(j) = false; % eliminar la de menor score
            end
        end
    end
    kept = boxes(keep, :);
end

```

```

function [iou, r1, r2] = getOverlap(A, B)
    x1 = max(A(1), B(1));
    y1 = max(A(2), B(2));
    x2 = min(A(1)+A(3), B(1)+B(3));
    y2 = min(A(2)+A(4), B(2)+B(4));

    iw = max(0, x2 - x1);
    ih = max(0, y2 - y1);
    intersection = iw * ih;

    aA = A(3) * A(4);
    aB = B(3) * B(4);
    union = aA + aB - intersection;

    iou = intersection / union;
    r1 = intersection / aA; % Fracción de A cubierta por B
    r2 = intersection / aB; % Fracción de B cubierta por A
end

```

5.1.3 plate_classifier

```

function [prediction, score] = classify_roi(img)

% Load model

persistent model;

```

```

    if isempty(model)

        data = load('plate_classifier.mat');

        model = data.model;

    end

    % Config

    imageSize = [64 128];

    % Features

    features = extract_plate_features(img, imageSize);

    % Prediction

    [prediction, scores] = predict(model, features);

    % Score

    score = scores(2);

end

function features = extract_plate_features(img, imageSize)

    % Grayscale

    if size(img,3) == 3
        img = rgb2gray(img);
    end

    % Aspect and Area Ratio

    [h, w] = size(img);
    aspectRatio = w / h;
    areaRatio = (h * w);

    % Resize

    img = imresize(img, imageSize);
    img = im2uint8(im2double(img));

```

```

    % HOG FEATURES

    hog_feature = extractHOGFeatures(img,'CellSize', [8 8]);

    features = [hog_feature, aspectRatio, areaRatio] ;

end

clear;
clc;
close all;

%% Configuration

positiveDir = 'dataset/positive';
negativeDir = 'dataset/negative';

imageSize = [64 128]; % height * width

%% Load dataset

positiveImages = dir(fullfile(positiveDir, '*.jpg'));
negativeImages = dir(fullfile(negativeDir, '*.jpg'));

numPos = length(positiveImages);
numNeg = length(negativeImages);

fprintf('Positives: %d\n', numPos);
fprintf('Negatives: %d\n', numNeg);

%% Features extraction

features = [];
labels = [];

% Positives

for i = 1:numPos

    imgPath = fullfile(positiveDir, positiveImages(i).name);

    img = imread(imgPath);

```

```

    feat = extract_plate_features(img, imageSize);

    features = [features; feat];

    labels = [labels; 1];

end

% Negatives

for i = 1:numNeg

    imgPath = fullfile(negativeDir, negativeImages(i).name);

    img = imread(imgPath);

    feat = extract_plate_features(img, imageSize);

    features = [features; feat];

    labels = [labels; 0];

end

fprintf('Features extracted\n');
size(features)

%% Train / Test division

rng(42);
cv = cvpartition(labels, 'HoldOut', 0.2);

Xtrain = features(training(cv), :);
Ytrain = labels(training(cv));

Xtest = features(test(cv), :);
Ytest = labels(test(cv));

%% Train classifier (SVM)

fprintf('Training SVM...\n');

model = fitcsvm(Xtrain, Ytrain, 'KernelFunction', 'linear', ...
    'Standardize', true, 'ClassNames', [0 1], 'KernelScale', 'auto');

```

```

fprintf('Model trained\n');

%% Evaluation

predictions = predict(model, Xtest);

accuracy = sum(predictions == Ytest) / length(Ytest);

fprintf('Accuracy: %.2f %%\n', accuracy * 100);

%% Confusion matrix

figure;
confusionchart(Ytest, predictions);

title('Confusion Matrix');

%% Save model

save('plate_classifier.mat', 'model');

fprintf('Model saved in plate_classifier_model.mat\n');

```

5.1.4 character_segmentation

```

function [alignedCrop, alignedBB] = alignPlate(im, boundingBox)
    if size(im, 3) == 3
        im = rgb2gray(im);
    end
    bw = imadjust(im);
    bw = imbinarize(bw, "adaptive");
    [szy, szx] = size(im);
    alignedBB = boundingBox;

    CC = bwconncomp(bw);
    S = regionprops(CC, 'Orientation', 'Area', 'Extent');
    [~, idx] = max([S.Area]);
    if ~isempty(S)
        ang = S(idx).Orientation;
        im = imrotate(im, -ang);
        bw = imrotate(bw, -ang);

        CC = bwconncomp(bw);
        S = regionprops(CC, 'BoundingBox', 'Area');
    end

```

```

[~, idx] = max([S.Area]);
bb = S(idx).BoundingBox;

[szyR, szxR] = size(im);

off = 15;
bx = max(1, bb(1) - off);
by = max(1, bb(2) - off);
bwi = min(szxR - bx, bb(3) + (2 * off));
bhe = min(szyR - by, bb(4) + (2 * off));
bb2 = [bx, by, bwi, bhe];

im = imcrop(im, bb);

rot = [
    bb(1), bb(2);
    bb(1) + bb(3), bb(2);
    bb(1) + bb(3), bb(2) + bb(4);
    bb(1), bb(2) + bb(4);
];

C0 = [szx, szy] / 2;
CR = [szxR, szyR] / 2;
R = [cosd(ang), -sind(ang); sind(ang), cosd(ang)];
corners = zeros(4, 2);
for i = 1:4
    pR = rot(i, :) - CR;
    p0 = pR * R;
    corners(i, :) = p0 + C0;
end

alignedBB = corners + [boundingBox(1), boundingBox(2)] - 1;

end
alignedCrop = im;
%figure, imshow(alignedCrop);
end

```

```

function chars = segm(im)

    if size(im, 3) == 3
        im = rgb2gray(im);
    end

```

```

im = imgaussfilt(im, 0.5);
bw = ~imbinarize(im, 'adaptive', 'ForegroundPolarity', 'dark', 'Sensitivity', 0

bw = imclearborder(bw); % Elimina objetos tocando los bordes
bw = bwareaopen(bw, 30);

CC = bwconncomp(bw);

% Close para conectar numeros como 3, etc
se = strel('square', 2);
bw = imclose(bw, se);

S = regionprops(CC, 'BoundingBox', 'Area', 'Extent');

[imgHeight, ~] = size(bw);
blobs = [];

for k = 1:numel(S)
    bb = S(k).BoundingBox;
    x = bb(1);
    y = bb(2);
    w = bb(3);
    h = bb(4);

    ar = w / h; % Aspect Ratio

    cond_ar = (ar > 0.05 && ar < 1.5);

    cond_h = (h > imgHeight * 0.35 && h < imgHeight * 0.95);

    % Extent (Area del objeto / Area del Bounding Box)
    cond_ext = (S(k).Extent > 0.12 && S(k).Extent < 1.0);

    if cond_ar && cond_h && cond_ext
        dfW = w * 0.10;
        dfH = h * 0.05;
        x2 = x - dfW;
        y2 = y - dfH;
        w2 = w + dfW * 2;
        h2 = h + dfH * 2;
        blobs = [blobs; [x2, y2, w2, h2]];
    end
end

% Para descartar simbolos/banderas

```

```

    if size(blobs, 1) >= 3
        medH = median(blobs(:, 4));
        keepMask = blobs(:, 4) > medH * 0.80;
        blobs = blobs(keepMask, :);
    end

    if ~isempty(blobs)
        chars = sortrows(blobs, 1);
    else
        chars = [];
    end
end
end

```

5.1.5 dataset_generation

```

% Intersection over union
% Checks how overlapped is the negative crop compared with the license
% plate

```

```

function iou = compute_iou(boxA, boxB)

    xA = max(boxA(1), boxB(1));
    yA = max(boxA(2), boxB(2));

    xB = min(boxA(1)+boxA(3), boxB(1)+boxB(3));
    yB = min(boxA(2)+boxA(4), boxB(2)+boxB(4));

    interW = max(0, xB - xA);
    interH = max(0, yB - yA);

    interArea = interW * interH;

    areaA = boxA(3) * boxA(4);
    areaB = boxB(3) * boxB(4);

    unionArea = areaA + areaB - interArea;

    iou = interArea / unionArea;

end

```

```

clear;
clc;

```

```

close all;

%% Configuration

imagesDir = 'images';
annotationsDir = 'annotations';

positiveDir = 'dataset/positive';
negativeDir = 'dataset/negative';

mkdir(positiveDir);
mkdir(negativeDir);

padding = 0.05; % 5%

negativePerImage = 5;

%% Read Images

imageFiles = dir(fullfile(imagesDir, '*.png'));

positiveCount = 1;
negativeCount = 1;

%% Processing

for i = 1:length(imageFiles)

    fprintf('Processing image %d / %d\n', i, length(imageFiles));

    % Read Image

    imageName = imageFiles(i).name;

    imgPath = fullfile(imagesDir, imageName);

    img = imread(imgPath);

    [imgH, imgW, ~] = size(img);

    % Read XML

    xmlName = replace(imageName, '.png', '.xml');

    xmlPath = fullfile(annotationsDir, xmlName);

```

```

xmlDoc = xmlread(xmlPath);

objects = xmlDoc.getElementsByTagName('object');

if objects.getLength == 0
    continue;
end

% Read only license plate

obj = objects.item(0);

bbox = obj.getElementsByTagName('bndbox').item(0);

xmin = str2double(bbox.getElementsByTagName('xmin').item(0).getTextContent);
ymin = str2double(bbox.getElementsByTagName('ymin').item(0).getTextContent);
xmax = str2double(bbox.getElementsByTagName('xmax').item(0).getTextContent);
ymax = str2double(bbox.getElementsByTagName('ymax').item(0).getTextContent);

% Add padding

bw = xmax - xmin;
bh = ymax - ymin;

padX = round(bw * padding);
padY = round(bh * padding);

xminP = max(1, xmin - padX);
yminP = max(1, ymin - padY);

xmaxP = min(imgW, xmax + padX);
ymaxP = min(imgH, ymax + padY);

% Positive crop

positiveCrop = img(yminP:ymaxP, xminP:xmaxP, :);

positiveName = sprintf('pos_%05d.jpg', positiveCount);

positiveCrop = im2uint8(mat2gray(positiveCrop));

imwrite(positiveCrop, fullfile(positiveDir, positiveName));

```

```

positiveCount = positiveCount + 1;

% Generate negatives (no plate)

for n = 1:negativePerImage

    validCrop = false;

    attempts = 0;

    while ~validCrop && attempts < 100

        attempts = attempts + 1;

        % Similar size to the license plate

        cropW = bw;
        cropH = bh;

        rx = randi([1, imgW - cropW]);
        ry = randi([1, imgH - cropH]);

        % IOU

        iou = compute_iou([rx ry cropW cropH], [xmin ymin bw bh]);

        if iou < 0.1

            negativeCrop = img(ry:ry+cropH, rx:rx+cropW, :);

            negativeName = sprintf('neg_%05d.jpg', negativeCount);

            negativeCrop = im2uint8(mat2gray(negativeCrop));

            imwrite(negativeCrop, fullfile(negativeDir, negativeName));

            negativeCount = negativeCount + 1;

            validCrop = true;

        end

    end

end

end

```

```
end
```

```
fprintf('Dataset successfully generated\n');
```

5.1.6 ocr_classifier

```
function label = classify_character(img)
```

```
    % Load model
```

```
    persistent svmModel5;
```

```
    if isempty(svmModel5)
```

```
        data = load('ocr_model5.mat');
```

```
        svmModel5 = data.svmModel5;
```

```
    end
```

```
    % Config
```

```
    imgSize = [32 32];
```

```
    % Extract Features
```

```
    features = extract_character_features(img, imgSize);
```

```
    % Classify
```

```
    label = predict(svmModel5, features);
```

```
end
```

```
function features = extract_character_features(img, imageSize)
```

```
    if size(img,3) == 3
```

```
        img = rgb2gray(img);
```

```
    end
```

```
    img = imresize(img, imageSize);
```

```
    img = im2single(img);
```

```

bin = imbinarize(img);
se = strel('disk', 1);
bin = imclose(bin, se);
bin = bwareaopen(bin, 5);

% 1. HOG
hog_feature = extractHOGFeatures(img, 'CellSize', [4 4]);

% 2. Density global
density = sum(bin(:)) / numel(bin);

% 3. Número de agujeros
cc = bwconncomp(~bin);
numHoles = cc.NumObjects;

% 4. Perfiles de proyección completos (16+16)
targetLen = 16;
hProfile = sum(bin, 1);
vProfile = sum(bin, 2);
hProf_norm = imresize(hProfile / (max(hProfile) + eps), [1, targetLen]);
vProf_norm = imresize(vProfile' / (max(vProfile) + eps), [1, targetLen]);
projFeatures = [hProf_norm, vProf_norm];

% 5. Zoning 4x4 (densidad por zona)
zoningF = zoningFeatures(bin, 4, 4);

% 6. Zoning asimétrico 6x3
zoningF2 = zoningFeatures(bin, 6, 3);

% 7. Crossing numbers por fila
crossings = zeros(1, targetLen);
binResized = imresize(bin, [targetLen, targetLen]) > 0.5;
for row = 1:targetLen
    line = binResized(row, :);
    crossings(row) = sum(diff(line) == 1);
end
crossings = crossings / (max(crossings) + eps);

features = [
    hog_feature, ...
    density, ...
    numHoles, ...
    projFeatures, ...
    zoningF, ...
    zoningF2, ...
    crossings

```

```

    ];
end

function zf = zoningFeatures(bin, rows, cols)
    [H, W] = size(bin);
    zf = zeros(1, rows * cols);
    idx = 1;
    for r = 1:rows
        for c = 1:cols
            rStart = round((r-1)*H/rows) + 1;
            rEnd    = round(r*H/rows);
            cStart = round((c-1)*W/cols) + 1;
            cEnd    = round(c*W/cols);
            zone = bin(rStart:rEnd, cStart:cEnd);
            zf(idx) = sum(zone(:)) / numel(zone);
            idx = idx + 1;
        end
    end
end
end

```

```

%function features = extract_character_features(img, imageSize)

% Preprocess

%if size(img,3) == 3
%    img = rgb2gray(img);
%end

% img = imresize(img, imageSize);

```

```

% img = im2single(img);

% BINARIZATION (para shape features)
% bin = imbinarize(img);

% HOG FEATURES (base)
% hog_feature = extractHOGFeatures(img, 'CellSize', [8 8]);

% 1. SHAPE FEATURES (muy importantes para 8 vs B, 1 vs I)
% density = sum(bin(:)) / numel(bin);

% agujeros (topología básica)
% cc = bwconncomp(~bin);
% numHoles = cc.NumObjects;

% =====
% 2. PROJECTION FEATURES (OCR clásico potente)
% =====

% hProfile = sum(bin, 1);
% vProfile = sum(bin, 2);

% projFeatures = [
%     mean(hProfile), std(hProfile), ...
%     mean(vProfile), std(vProfile)
% ];

% =====
% 3. GRADIENT FEATURES (complemento a HOG)
% =====

% [Gx, Gy] = imgradientxy(img);
% [Gmag, Gdir] = imgradient(Gx, Gy);

% gradFeatures = [
%     mean(Gmag(:)), ...
%     std(Gmag(:))
% ];

% dirHist = histcounts(Gdir(:), 8);
% dirHist = dirHist / (sum(dirHist) + eps);

% =====

```

```

    % FINAL FEATURE VECTOR
    % =====
    % features = [
    %     hog_feature, ...
    %     density, ...
    %     numHoles, ...
    %     projFeatures, ...
    %     gradFeatures, ...
    %     dirHist
    % ];

%end

%function features = extract_character_features(img, imageSize)

    % Grayscale

    %if size(img,3) == 3
        %img = rgb2gray(img);
    %end

    % Aspect and Area Ratio

    %[h, w] = size(img);
    %aspectRatio = w / h;
    %areaRatio = (h * w);

    % Connected components

    %bw = imbinarize(img);
    %cc = bwconncomp(bw);
    %numComponents = cc.NumObjects;

    % Resize

    %img = imresize(img, imageSize);
    %img = im2uint8(im2double(img));

    % HOG FEATURES

    %hog_feature = extractHOGFeatures(img,'CellSize', [8 8]);

    %features = [hog_feature];

%end

```

```

function [plateText, charLabels] = recognize_plate(aligned, blobs)

    plateText = "";
    charLabels = strings(size(blobs,1),1);

    for k = 1:size(blobs,1)

        bb = blobs(k,:);

        charImg = imcrop(aligned, bb);

        predictedChar = classify_character(charImg);

        charLabels(k) = string(predictedChar);

        plateText = plateText + charLabels(k);

    end
end

clear;
clc;
close all;

%% Configuration

datasetPath = fullfile(pwd, "Datasets/charactersDataset2/training_data");

imds = imageDatastore(datasetPath, ...
    "IncludeSubfolders", true, ...
    "LabelSource", "foldernames");

[imdsTrain, imdsTest] = splitEachLabel(imds, 0.8, "randomized");

%% Features Extraction and Model Train

numTrain = numel(imdsTrain.Files);
numTest  = numel(imdsTest.Files);

XTrain = [];
YTrain = imdsTrain.Labels;

```

```

XTest = [];
YTest = imdsTest.Labels;

imgSize = [32 32];

% TRAIN FEATURES

for i = 1:numTrain

    img = readimage(imdsTrain, i);

    feat = extract_character_features(img, imgSize);

    XTrain = [XTrain; feat];

end

% TEST FEATURES

for i = 1:numTest

    img = readimage(imdsTest, i);

    feat = extract_character_features(img, imgSize);

    XTest = [XTest; feat];

end

%% Train model

svmModel5 = fitcecoc(XTrain, YTrain);

%% Model Evaluation

YPred = predict(svmModel5, XTest);

accuracy = sum(YPred == YTest) / numel(YTest);

fprintf("Accuracy: %.2f%%\n", accuracy * 100);

figure;

```

```
confusionchart(YTest, YPred);  
title("Confusion Matrix - OCR SVM");  
  
save("ocr_model5.mat", "svmModel5", "imgSize");
```